LAPORAN PRATIKUM PEMROGRAMAN BERBASIS OBJEK

Transaksi Dinamis (Polymorphism)

Pekan 5

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : NURFIAH, S.ST, M.KOM

ASISTEN LABORATORIUM : Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

ISI

A. Langkah Kerja Praktikum

Langkah 1 : Refaktor Method Tarik Tunai pada Kelas Induk (Rekening.java)

1. Agar fungsi penarikan dapat digunakan sebagai bagian dari fitur transfer dan memberikan status keberhasilan, tipe kembalian (*return type*) method tarik diubah dari void menjadi boolean. Method ini akan mengembalikan true jika penarikan sukses, dan false jika gagal (misalnya, saldo tidak cukup).

   Potongan Kode:

```java
// Langkah 1: return type boolean, true jika sukses, false jika gagal
public boolean tarik(double nominal) {
    if (nominal > saldo) {
        System.out.println("Saldo tidak cukup");
        return false;
    }
    debit(nominal, "Tarik Tunai");
    return true;
}
```

Langkah 2 : Melakukan Overriding pada RekeningGiro.java

1. Pada kelas anak RekeningGiro, method tarik ditimpa (*overridden*) menggunakan anotasi @Override sebagai praktik terbaik (*best practice*). Logika penarikan dimodifikasi agar mengizinkan saldo minus selama nominal penarikan masih berada dalam batas saldo + batasOverdraft.

   Potongan Kode:

```java
// Langkah 2: Overriding agar Giro mengizinkan saldo minus hingga batas overdraft
@Override
public boolean tarik(double nominal) {
    if (nominal > getSaldo() + batasOverdraft) {
        System.out.println("Saldo dan limit overdraft tidak mencukupi!");
        return false;
    }
    debit(nominal, "Tarik Tunai (Giro)");
    return true;
}
```

Langkah 3: Uji Coba Pengikatan Dinamis (*Dynamic Method Dispatch*)

1. Pada Main.java, saat pengguna memilih menu Tarik Tunai (Menu 3), program memanggil akunAktif.tarik(nominalTarik). Berkat *Dynamic Method Dispatch* (Late Binding), JVM secara otomatis menentukan method mana yang akan dieksekusi pada saat *runtime*. Jika akunAktif adalah objek RekeningGiro, JVM akan memanggil logika tarik yang telah di-*override* di RekeningGiro tanpa memerlukan struktur if-else atau switch-case untuk mengecek tipe objek secara eksplisit.

B. Latihan dan Tugas

Tugas 1: Pembuatan Multi-Produk (40%)

Tugas: Fitur Transfer Antar Rekening

Keindahan polimorfisme diimplementasikan dalam fitur transfer, di mana sistem dapat menerima objek tujuan dari berbagai subclass (RekeningTabungan atau RekeningGiro) melalui satu antarmuka parameter yang seragam (Rekening).

1. Pembuatan Method Transfer di Kelas Induk

Di dalam class Rekening.java, ditambahkan method transfer yang menerima parameter polimorfik Rekening tujuan. Logika method ini terdiri dari tiga tahap:

- Otentikasi: Memverifikasi pinInput. Jika salah, proses dibatalkan.
- Penarikan Dinamis: Memanggil this.tarik(nominal). Jika objek saat ini adalah RekeningGiro, ia akan otomatis menggunakan aturan *overdraft* berkat *Dynamic Dispatch*.
- Penyetoran: Jika penarikan bernilai true, method tujuan.setor(nominal) dipanggil untuk menambah saldo rekening tujuan.

Potongan Kode:

```
// Tugas: Fitur Transfer dengan parameter polimorfik (bisa Tabungan atau Giro)
public void transfer(Rekening tujuan, double nominal, String pinInput) {
    // Pertama: otentikasi PIN, jika false batalkan
    if (!otentikasi(pinInput)) {
        System.out.println("Transfer dibatalkan: PIN yang Anda masukkan salah!");
        return;
    }

    // Kedua: Dynamic Dispatch, jika this adalah Giro otomatis memakai aturan overdraft
    if (this.tarik(nominal)) {
        // Ketiga: jika penarikan sukses, setor ke rekening tujuan
        tujuan.setor(nominal);
        System.out.println("Transfer sebesar Rp " + nominal + " ke rekening "
                + tujuan.getNomorRekening() + " (" + tujuan.getNamaPemilik() + ") berha
    } else {
        System.out.println("Transfer gagal: penarikan dana ditolak.");
    }
}
```

2. Integrasi ke CLI Utama (Main.java)

   Menu baru "8. Transfer ke Rekening Lain" ditambahkan ke dalam *Command Line Interface* (CLI). Program meminta pengguna memasukkan nomor rekening tujuan, kemudian melakukan pencarian (*linear search*) di dalam ArrayList<Rekening> daftarRekening. Jika ditemukan dan bukan rekening sendiri, program meminta nominal dan PIN, lalu mengeksekusi method transfer.

   Potongan Kode:

```
case 8:
    // Menu 8: Transfer ke Rekening Lain
    if (akunAktif == null) {
        System.out.println("Belum ada rekening yang dibuat!");
        break;
    }

    System.out.print("Masukkan Nomor Rekening Tujuan: ");
    String nomorTujuan = scanner.nextLine();

    // Cari nomor tersebut di dalam daftarRekening
    Rekening akunTujuan = null;
    for (Rekening r : daftarRekening) {
        if (r.getNomorRekening().equals(nomorTujuan)) {
            akunTujuan = r;
            break;
        }
    }

    if (akunTujuan == null) {
        System.out.println("Rekening dengan nomor " + nomorTujuan + " tidak ditemukan!
        break;
    }
    if (akunTujuan == akunAktif) {
        System.out.println("Gagal: Tidak bisa transfer ke rekening sendiri!");
        break;
    }

    System.out.print("Masukkan nominal transfer: ");
    double nominalTransfer = scanner.nextDouble();
    scanner.nextLine(); // Clear buffer

    System.out.print("Masukkan PIN Anda: ");
    String pinTransfer = scanner.nextLine();

    // Parameter polimorfik: akunTujuan bisa Tabungan atau Giro
    akunAktif.transfer(akunTujuan, nominalTransfer, pinTransfer);
    break;
```

C. Link Repository GitHub :
https://github.com/kar1mmmm/PrakPBO_C_2511532029